

Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: IMPLEMENTACIÓN DE ANALIZADORES LÉXICOS EN LEX.

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

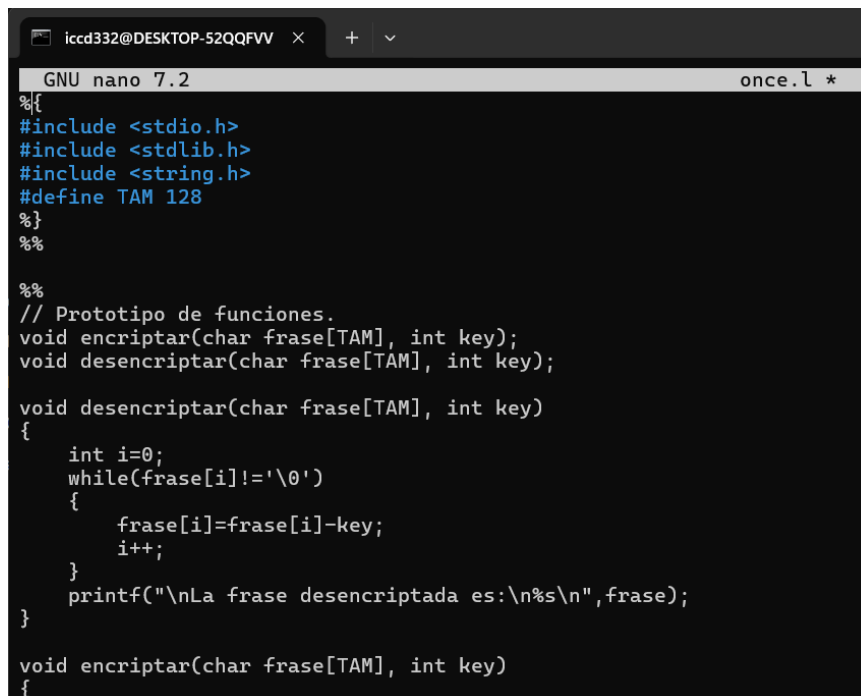
1. Resumen

El presente ejercicio consiste en la implementación del clásico algoritmo de cifrado César. El programa recibe una cadena de texto y un número entero ingresados por el usuario, procediendo a desplazar cada carácter en la tabla ASCII según el valor de la llave para encriptar el mensaje, y realizando el proceso aritmético inverso para desencriptarlo.

2. Estructura de un Archivo LEX

Un archivo fuente lex se encuentra estructurado por los siguientes bloques funcionales separados por (%%):

- **Área de declaración de variables:** Se declaran las librerías y variables que se van a utilizar.
- **Área de expresiones regulares:** En esta se adjuntan todas las expresiones regulares necesarias para cada programa y su respectiva impresión en consola si se llega a encontrar algo relacionado con la misma.
- **Área de código:** En esta se empieza a codificar en el lenguaje previamente seleccionado.



```
iccd332@DESKTOP-52QQFV  x  +  v
GNU nano 7.2                                once.l *
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define TAM 128
%}
%%

%%
// Prototipo de funciones.
void encriptar(char frase[TAM], int key);
void desencriptar(char frase[TAM], int key);

void desencriptar(char frase[TAM], int key)
{
    int i=0;
    while(frase[i]!='\0')
    {
        frase[i]=frase[i]-key;
        i++;
    }
    printf("\nLa frase desencriptada es:\n%s\n",frase);
}

void encriptar(char frase[TAM], int key)
{
```

```
iccd332@DESKTOP-52QQFVV x + v
GNU nano 7.2
while(frase[i]!='\0')
{
    frase[i]=frase[i]+key;
    i++;
}
| printf("\nLa frase encriptada es:\n%s\n",frase);
}

// Procedimiento principal.
int main()
{
    int x;
    char cadena[TAM];
    printf("introduce una cadena:      ");
    scanf("%128[^\n]", cadena);
    while(getchar()!='\n');
    printf("introduce la llave para encriptar : ");
    scanf("%d",&x);
    while(getchar()!='\n');

    encriptar(cadena, x);
    getchar(); // Pausa

    desencriptar(cadena, x);
    getchar(); // Pausa
```

```
return 0;
}
```

```
iccd332@DESKTOP-52QQFVV x + v
(base) iccd332@DESKTOP-52QQFVV:~$ mkdir once
(base) iccd332@DESKTOP-52QQFVV:~$ cd once
(base) iccd332@DESKTOP-52QQFVV:~/once$ nano once.l
(base) iccd332@DESKTOP-52QQFVV:~/once$ lex once.l
(base) iccd332@DESKTOP-52QQFVV:~/once$ gcc lex.yy.c -lfl
(base) iccd332@DESKTOP-52QQFVV:~/once$ ./a.out
introduce una cadena:      HOLA
introduce la llave para encriptar : 13

La frase encriptada es:
U\YN

La frase desencriptada es:
HOLA
(base) iccd332@DESKTOP-52QQFVV:~/once$ |
```

3. Conclusión

La práctica evidencia que la estructura de un archivo Lex es sumamente flexible, permitiendo la declaración y ejecución de algoritmos procedimentales tradicionales como el Cifrado César mediante manipulación ASCII.